

If you are experiencing TWiki instability from outside the CERN network, please connect via the CERN internal network - see [KB0010688](#) for details

[TWiki](#) > [CMSPublic Web](#) > [SWGuide](#) > [SWGuideVertexReco](#) > [SWGuideKinematicVertexFit](#) (2013-02-27, FrankMeier)

Kinematic Vertex Fit

Complete: 4

Contents:

- [Overview](#)
- [Documentation](#)
- [The framework, and usage guide](#)
- [The KinematicParticle and KinematicVertex](#)
- [The KinematicTree](#)
- [Application of constraints](#)
 - [The global fit](#)
 - [The sequential fit](#)
- [Constraints](#)
 - [For the global fit](#)
 - [For the sequential fit](#)
- [Some additional comments](#)
- [Various Tools](#)
- [Bug fixes and tags to use](#)
 - [Usage in CMSSW 2.0.X, 2.1.X and 2.2X releases](#)
 - [Usage in CMSSW 1.6.X releases](#)
 - [Usage in CMSSW 1.3.X releases](#)

Overview

The goal of a kinematic fit is to improve the resolution of experimental measurements, test hypothesis and to find unknown parameters by introducing constraints derived from physics laws into the minimization problem. It is based on a Least Means Squared minimization with Lagrange multipliers.

The kinematic fit package provides a flexible framework with generic minimization algorithms which do not depend on the constraints, such that the constraint can be chosen by the user. The kinematic fit package also allows the full decay chain reconstruction according to user-defined decay model. The full decay tree is reconstructed by fitting all tracks and vertices in current decay and application of user-defined constraints on their parameters.

The fitter is implemented in two packages, [RecoVertex/KinematicFitPrimitives](#) and [RecoVertex/KinematicFit](#).

Documentation

The Kinematic fit has been described extensively in a [CHEP conference proceeding](#) and a CMS analysis note:

Prokofiev, K. and Speer, T., "A Kinematic fit and a decay chain reconstruction library", [CMS Internal Note, CMS-IN-2004/020](#).

The framework, and usage guide

The full example is in [KineExample](#), and the cfg is in [here](#).

The KinematicParticle and KinematicVertex

During the kinematic fit the particles and the reconstructed vertices are represented by the KinematicParticle and the KinematicVertex classes respectively.

The [KinematicParticle](#) class stores the reconstructed state of the trajectory, assigned or reconstructed mass, the corresponding covariance matrix, the charge, the χ^2 and the number of degrees of freedom assigned to the current particle during previous stages of the reconstruction.

In the present version of the library, a state of the particle is described by 7 parameters:

- (x,y,z) : a reference position in the global frame
- (p_x,p_y,p_z) : the momentum of the particle at this point
- m : the mass of the particle, calculated as a result of previous fits or hypothesis assigned to the final state track

The vector of 7 parameters and their joint (7x7) covariance matrix are cached in the [KinematicState](#) class. The state of the KinematicParticle after the last refit is accessible via the KinematicState `KinematicParticle::currentState()` method. The KinematicParticle can be created as a result of the kinematic fit or via a special factory, assigning a mass hypothesis to the final state trajectory, the [KinematicParticleFactoryFromTransientTrack](#), for example.

The [KinematicVertex](#) class stores the reconstructed vertex position, its corresponding covariance matrix, the χ^2 and the number of degrees of freedom assigned to the vertex during the last refit. The KinematicVertex vertex is always created as a result of the vertex fit.

Both KinematicParticle and KinematicVertex classes store the pointer to the last constraint applied and the state of the particle or vertex before that constraint had been applied.

The following piece of code illustrates the production of KinematicParticles assigning the mass guess to the final state tracks.

```
//The reconstructed muon trajectories:
vector<TransientTrack> muonTracks;

//Creating a KinematicParticleFactory
KinematicParticleFactoryFromRecTrack pFactory;

//The mass of a muon and the insignificant mass sigma to avoid singularities in the covariance matrix.
ParticleMass muon_mass = 0.1056583;
float muon_sigma = 0.0000000001;

//initial chi2 and ndf before kinematic fits. The chi2 of the reconstruction is not considered
float chi = 0.;
float ndf = 0.;

//making particles
vector<RefCountedKinematicParticles> muonParticles;
for(vector<RecTrack>::const_iterator i = muonTracks.begin(); i != muonTracks.end();++i)
{ muonParticles.push_back(factory.particle (*i,muon_mass,chi,ndf,muon_sigma)); }
```

The KinematicTree

The structure of the reconstructed decay chain is cached in the [KinematicTree](#) class. This class contains pointers to all the KinematicParticles and KinematicVertices forming the decay chain and allows the user to navigate between them. A KinematicTree is created as a result of one or a series of the kinematic fits.

For example, to reconstruct the $B_s \rightarrow J/\Psi \Phi \rightarrow \mu^+ \mu^- K^+ K^-$ decay tree out of the 4 final state TransientTracks, the following steps should be performed:

- Create the vector of KinematicParticles, assigning the mass hypothesis to the final state particles, as shown in the previous section.
- Create an instance of the [KinematicParticleVertexFitter](#), which reconstructs the vertex out of a given set of KinematicParticles and calculates the parameters of the decayed state (more on fitters in the next section).
- Perform a fit and get the reconstructed tree via the following method

```
RefCountedKinematicTree KinematicParticleVertexFitter::fit(vector)
```

The reconstructed tree now contains:

- The final state of the particles refitted with the vertex constraint.
- The reconstructed vertex.
- The reconstructed mother particle, the B_s meson in this case.

The KinematicParticles and KinematicVertices stored in the tree can be accessed as shown below. It should be noted that when a KinematicTree is used in a subsequent fit (such as in the example of the [SequentialFit](#) below), the pointer in that KinematicTree is

moved. When you so which to access again information in that tree, you can not assume that the pointer is at the place you left it before the fit (it could be anywhere in the tree). You should therefore not forget to reset and move the pointer.

```
/**
 * The reconstructed decay tree is a result of the kinematic fit
 * The KinematicParticleVertexFitter fits the final state particles to their vertex and
 * reconstructs the decayed state (see next section)
 */
KinematicParticleVertexFitter fitter;
RefCountedKinematicTree myTree = fitter.fit(particles);

//accessing the tree components
myTree->movePointerToTheTop();

//We are now at the top of the decay tree getting the B_s reconstructed KinematicParticle
RefCountedKinematicParticle b_s = myTree->currentParticle();

//getting the B_s decay vertex
RefCountedKinematicVertex b_dec_vertex = myTree->currentDecayVertex();

//accessing the reconstructed Bs meson parameters:
AlgebraicVector bs_par = b_s->currentState().kinematicParameters().vector();

//and their joint covariance matrix:
AlgebraicMatrix bs_er = b_s->currentState().kinematicParametersError().matrix();

//Now navigating down the tree
bool child = myTree->movePointerToTheFirstChild();

if(child)
{
// access parameters the same way it was done for the Bs (getting the current state etc)
}
bool nextChild = myTree->movePointerToTheNextChild();

if(nextChild)
{
// access parameters
}
//and so on....
```

Application of constraints

For example, reconstructing the Bs decay we can require:

- The final state track come from the same vertex
- The invariant mass of two muons is equal to the mass of the J/Psi meson.

The global fit

During the reconstruction of the decay chain, the constraints can be applied in two different ways. The most intuitive way is the so-called *Global* strategy, when all additional constraints are applied simultaneously with the vertex fit. This special [KinematicConstrainedVertexFitter](#) is provided for this strategy. This fitter takes a vector of KinematicParticles and a pointer to the constraint class ([TwoTrackMassKinematicConstraint](#) in this case) as input and returns a reconstructed tree.

The following code example illustrates the reconstruction of the parameters of the Bs meson using the global strategy, requiring the invariant mass of 2 muons to be equal to the mass of the J/Psi meson:

```

//the final state muons and kaons from the Bs->J/PsiPhi->mumuKK decay
vector<RefCountedKinematicParticle> particles;

//creating the constraint for the J/Psi mass
ParticleMass jpsi = 3.09687;

//creating the two track mass constraint
MultiTrackKinematicConstraint * j_psi_c = new TwoTrackMassKinematicConstraint(jpsi);

//creating the fitter
KinematicConstrainedVetexFitter kcvFitter;

//obtaining the resulting tree
RefCountedKinematicTree myTree = kcvFitter.fit(particles, j_psi_c);

//accessing the refitted and reconstructed parameters (see the previous section)

```

Please note that the KinematicConstrainedVetexFitter already contains the vertex constraint, and so this constraint should NOT be included as a constraint in the fit method!

The sequential fit

However, a different (*Sequential*) strategy can be used.

- Fitting two final state muons to a common vertex, reconstructing thus the J/Psi parameters at this vertex.
- Constraining the invariant mass of two muons to be equal to the mass of the J/Psi
- Fitting the J/Psi and the two kaons to a common vertex, reconstructing the Bs parameters

This strategy can be implemented using two existing classes:

- The KinematicParticleVertexFitter fits the set of KinematicParticles to the joint vertex, reconstructs the parameters of the decayed state and returns the consistent decay tree, as shown above.
- The [KinematicParticleFitter](#) class refits the parameters of the top particle of the provided tree according to the given constraint ([MassKinematicConstraint](#) in this example).

The next example illustrates the use of Sequential strategy for the same decay:

```

//the final state muons and kaons produced by the KinematicParticleFactory:
vector<RefCountedKinematicParticle> muons;
vector<RefCountedKinematicParticle> kaons;

//creating the vertex fitter
KinematicParticleVertexFitter kpvFitter;

//reconstructing a J/Psi decay
RefCountedKinematicTree jpTree = kpvFitter.fit(muons);

//creating the particle fitter
KinematicParticleFitter csFitter;

// creating the constraint with a little sigma to put in the resulting covariance
// matrix in order to avoid singularities
ParticleMass jp = 3.09687;
float jp_m_sigma = 0.00004;
KinematicConstraint * jpsi_c = new MassKinematicConstraint(jp,jp_m_sigma);

//the constrained fit:
jpTree = csFitter.fit(jpsi_c,jpTree);

//getting the J/Psi KinematicParticle and putting it together with the kaons.
//The J/Psi KinematicParticle has a pointer to the tree it belongs to
jpTree->movePointerToTheTop();
RefCountedKinematicParticle jpsi_part = jpTree->currentParticle();
kaons.push_back(jpsi_part);

//making a vertex fit and thus reconstructing the Bs parameters
RefCountedKinematicTree bsTree = kpvFitter.fit(kaons);

// the resulting tree includes all the final state tracks, the J/Psi meson,
// its decay vertex, the Bs meson and its decay vertex.

```

Both strategies are mathematically identical and should yield the same result within the precision of the computations. The different reconstruction strategies together with details of implementation of the constraint classes are discussed in the corresponding CMS analysis.

Constraints

For the global fit

- Invariant mass constraint:
 - [TwoTrackMassKinematicConstraint](#) for two tracks
 - [MultiTrackMassKinematicConstraint](#) ( - from release 3.1 onwards) for any number of tracks.
- [ColinearityKinematicConstraint](#) ( - from release 3.1 onwards): Colinearity constraint for photons, in 2D or 3D
- [MultiTrackPointingKinematicConstraint](#) ( - NEW): Constraint to force the momentum vector to be aligned with the decay length vector. Can be used with arbitrary number of tracks.
- [CombinedKinematicConstraint](#) ( - NEW): Combines several constraints. Collect your constraints in a `std::vector<MultiTrackKinematicConstraint* >` and use this vector in the constructor of `CombinedKinematicConstraint`.

For the sequential fit

- [PointingKinematicConstraint](#): Constraint to force the momentum vector to be aligned with the decay length vector.

Some additional comments

- In the KineamticParticle, the currentState () gives you always the state with the already applied constraints (so the refitted state). The method initialState() returns the state with which the particle was first created. Beware that in the case of sequential fits, not all constraints are applied to all final particles. For example in the case B-> D0 l nu, D0->Kpi, where a first fit is made of the D0 vertex, and the D0 is then fit with the lepton to obtain the B vertex, the second vertex constraint can not be propagated to the two tracks from the D0 decay. These two states will thus be those obtained in the first fit only. The state of the D0 itself will be that obtained in the full fit.

Various Tools

The distance between two vertices can be computed using the usual VertexDistance tools, described [here](#). Simply use the vertexState() method to get the VertexState from the [KinematicVertex](#).

Bug fixes and tags to use

Usage in CMSSW 2.0.X, 2.1.X and 2.2X releases

A small bug was found which affects all the 2XY releases. Two packages have therefore to be checked-out from CVS into your area and compiled. The packages with the correct tags are:

- RecoVertex/KinematicFitPrimitives: V01-00-03
- RecoVertex/KinematicFit: V01-00-03

Usage in CMSSW 1.6.X releases

Several small bugs were found after the last 1.6 release. Two packages have therefore to be checked-out from CVS into your area and compiled. The packages with the correct tags are:

- RecoVertex/KinematicFitPrimitives: V00-03-03
- RecoVertex/KinematicFit: V00-00-04

Usage in CMSSW 1.3.X releases

Unfortunately, in the last releases of 1.3.X, several bug-fixes were not included. Three packages have therefore to be checked-out from CVS into your area, and compiled. The packages with the correct tags are:

- RecoVertex/VertexTools: V00-01-15
- RecoVertex/KinematicFitPrimitives: V00-00-02
- RecoVertex/KinematicFit: V00-00-02

```
scramv1 project CMSSW CMSSW_1_3_6
cd CMSSW_1_3_6/src/
cvs co -r V00-01-15 RecoVertex/VertexTools
cvs co -r V00-00-02 RecoVertex/KinematicFitPrimitives
cvs co -r V00-00-02 RecoVertex/KinematicFit
cd RecoVertex/
scramv1 b -j 4
cd KinematicFit/test/
eval `scramv1 runtime -csh`
cmsRun testKineFit.cfg
```

Review Status

Editor/Review and date	Comments
Main.speer - 05 Jul 2007	Major content addition

Responsible: Main.speer (Thomas Speer)
Last reviewed by:

I	Attachment	History	Action	Size	Date	Who	Comment
	CMS_IN_2004_20.pdf	r1	manage	217.0 K	2007-07-18 - 10:51	ThomasSpeer	

Topic revision: r28 - 2013-02-27 - [FrankMeier](#)

Copyright &© 2008-2026 by the contributing authors. All material on this collaboration platform is the property of the contributing authors.
or Ideas, requests, problems regarding TWiki? use [Discourse](#) or [Send feedback](#)

